EC382 EMBEDDED SYSTEM LABORATORY

Laboratory Project Report

WiFi-Based Activity Detection
Using CSI Data with ESP32

Student(s):	**Anubhav Anand (2301036)**
	Anurag Sahoo (2301039)
Department:	Electronics & Communication Engineering
Institution:	Indian Institute of Information Technology Guwahati
Section:	EC31
Session:	B.Tech Semester 6

Contents

1. Abstract

In this project, we explored how WiFi signals can be used to detect human activity without needing physical sensors like cameras or PIRs. We utilized Channel State Information (CSI), which provides granular data on how WiFi waves scatter and fade in a given environment. We configured two ESP32 boards on a local hotspot: one transmitting UDP broadcast packets at 50Hz, and the other sniffing packets in promiscuous mode to extract CSI amplitude. Instead of just looking at raw amplitude, our receiver calculates the temporal variance of 52 subcarriers over a sliding window. This data is sent via Serial to a PC running a Flask web server, which streams the metrics to a real-time Chart.js dashboard using WebSockets. When the variance ratio exceeds our defined threshold (1.8), motion is successfully detected and logged.

2. System Level Design & Flowchart

The system architecture consists of a hardware processing layer (two ESP32s and a smartphone hotspot) and a PC-based software visualization and debouncing layer. The hardware sniffs the wireless link to extract CSI subcarrier information, processes it into a variance metric, and streams it over serial for debouncing, event logging, and real-time dashboard plotting.

2.1 System Block Diagram

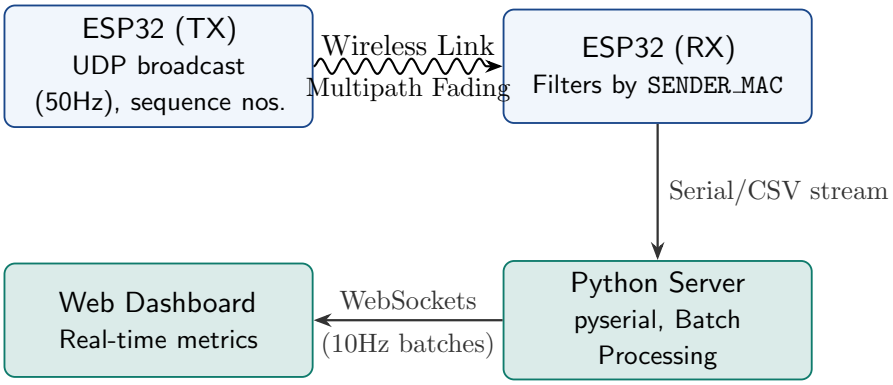

Figure 2.1: System Architecture Block Diagram

2.2 Software Execution Flowchart

The flowchart below reflects the exact processing steps in the C firmware and Python scripts. Overlaps have been eliminated for clarity.

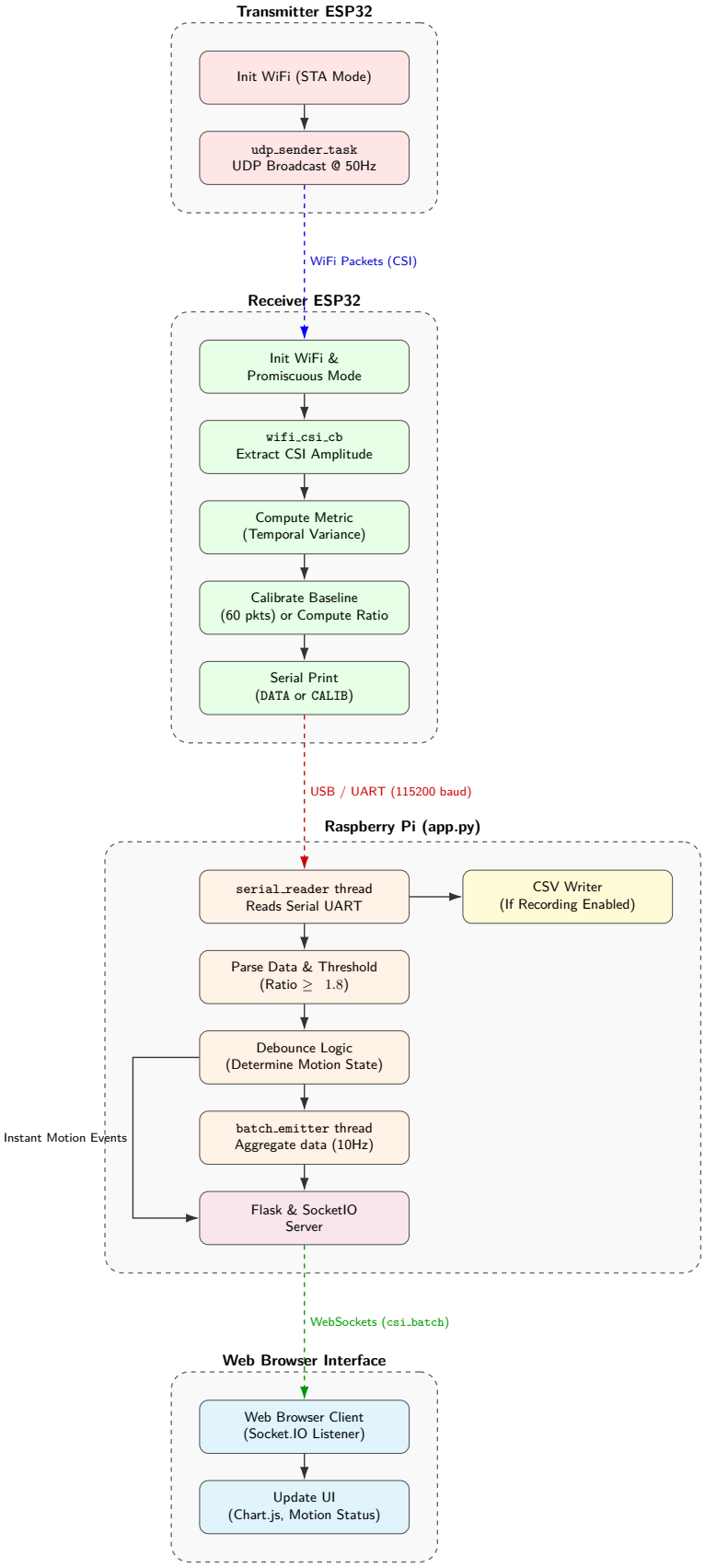

Figure 2.2: Software Execution Flowchart

3. Sensors & Subsystems Used

Component / Software	Description / Role
ESP32 DevKit v1 (×2)	Serves as the WiFi TX (UDP sender) and RX (Promiscuous sniffer) nodes. The onboard WiFi antenna acts as our primary sensor.
Mobile Hotspot	A Google Pixel 8 was used to create the local WLAN network to link the ESP32s.
esp-idf & esp-csi API	Used to extract the lower-layer MAC packets and access raw I/Q signal data from the baseband processor.
Python 3 (Flask)	Backend server running on the laptop. Uses `pyserial` for COM port reading.
Socket.IO & Chart.js	Frontend technologies used to display real-time, low-latency graphs without reloading the browser page.

Table 3.1: Hardware and Software Stack

4. Schematics

Since we are using the onboard WiFi antennas to gather data, there is absolutely no external wiring, breadboarding, or physical sensors required for this project. The hardware setup consists purely of power and data connections:

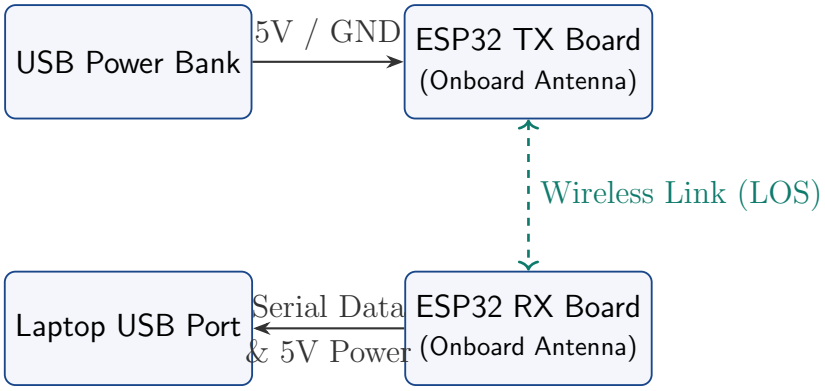

Figure 4.1: Hardware Setup Schematic

5. Step-by-Step Implementation

1. **Network Setup:** Used a 2.4GHz WiFi hotspot. Hardcoded its credentials into both `csi_sender.c` and `csi_receiver_serial.c`.
2. **Flashing hardware:** Flashed the sender code to ESP32 #1, which immediately started broadcasting UDP packets at 50Hz. Flashed the receiver code to ESP32 #2.
3. **Calibration Phase:** Powered both devices. The receiver code requires 60 packets in a completely still environment to calculate the quiet `baseline`. We stood still until the serial monitor printed "Calibrated!".
4. **Dashboard Launch:** Started `app.py` on the PC. It connected to `COM4` at 115200 baud. Opened `http://localhost:5000` in a web browser to view the real-time UI.
5. **Testing:** Walked through the Line-of-Sight between the two ESP32 nodes. Observed the variance ratio jump well past the 1.8 threshold, triggering the red "MOTION" alert on the dashboard and updating the event log.
6. **Data Collection:** Used the custom "Start Recording" UI button to log the timestamps, ratios, and motion flags into a CSV file for post-processing.

6. Source Codes (All Codes)

This section contains the entirety of the firmware and software stack used in the project.

6.1 ESP32 Transmitter Code (csi_sender.c)

```c
#include <stdio.h>
#include <string.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/event_groups.h"
#include "esp_wifi.h"
#include "esp_event.h"
#include "nvs_flash.h"
#include "esp_log.h"
#include "esp_netif.h"
#include "lwip/sockets.h"
#include "esp_timer.h"   // add at the top

#define WIFI_SSID        "Google Pixel 8"
#define WIFI_PASS        "biteload"
#define SEND_PORT        3333
#define SEND_INTERVAL_MS 20    // 50 packets/sec
#define TAG              "CSI_SENDER"

static EventGroupHandle_t s_wifi_event_group;
#define WIFI_CONNECTED_BIT BIT0

static esp_timer_handle_t s_reconnect_timer = NULL;

static void reconnect_timer_cb(void *arg) {
    ESP_LOGI(TAG, "Retrying connection...");
    esp_wifi_connect();
}

static void wifi_event_handler(void *arg, esp_event_base_t event_base,
                               int32_t event_id, void *event_data) {
    if (event_base == WIFI_EVENT && event_id == WIFI_EVENT_STA_START) {
        esp_wifi_connect();
    } else if (event_base == WIFI_EVENT && event_id == WIFI_EVENT_STA_DISCONNECTED) {
        wifi_event_sta_disconnected_t *disc = (wifi_event_sta_disconnected_t
            *)event_data;
        ESP_LOGW(TAG, "Disconnected, reason=%d. Retrying in 5s...", disc->reason);
        // Delay before retry - prevents Android hotspot from blacklisting the MAC
        esp_timer_start_once(s_reconnect_timer, 5000000);
        // 5 seconds in microseconds
        xEventGroupClearBits(s_wifi_event_group, WIFI_CONNECTED_BIT);
    } else if (event_base == IP_EVENT && event_id == IP_EVENT_STA_GOT_IP) {
        ip_event_got_ip_t *event = (ip_event_got_ip_t *)event_data;
        ESP_LOGI(TAG, "Got IP: " IPSTR, IP2STR(&event->ip_info.ip));
        xEventGroupSetBits(s_wifi_event_group, WIFI_CONNECTED_BIT);
        // Cancel any pending reconnect since we're now connected
        esp_timer_stop(s_reconnect_timer);
    }
}

static void wifi_init_sta(void) {
    s_wifi_event_group = xEventGroupCreate();
    ESP_ERROR_CHECK(esp_netif_init());
    ESP_ERROR_CHECK(esp_event_loop_create_default());
    esp_netif_create_default_wifi_sta();
```

```
55
56      wifi_init_config_t cfg = WIFI_INIT_CONFIG_DEFAULT();
57      ESP_ERROR_CHECK(esp_wifi_init(&cfg));
58
59      // Create the reconnect timer BEFORE registering event handlers
60      const esp_timer_create_args_t timer_args = {
61          .callback = reconnect_timer_cb,
62          .name     = "wifi_reconnect"
63      };
64      ESP_ERROR_CHECK(esp_timer_create(&timer_args, &s_reconnect_timer));
65
66      esp_event_handler_instance_t inst_any, inst_ip;
67      ESP_ERROR_CHECK(esp_event_handler_instance_register(
68          WIFI_EVENT, ESP_EVENT_ANY_ID, &wifi_event_handler, NULL, &inst_any));
69      ESP_ERROR_CHECK(esp_event_handler_instance_register(
70          IP_EVENT, IP_EVENT_STA_GOT_IP, &wifi_event_handler, NULL, &inst_ip));
71
72      wifi_config_t wifi_config = {
73          .sta = {
74              .ssid     = WIFI_SSID,
75              .password = WIFI_PASS,
76              .threshold.authmode = WIFI_AUTH_WPA2_PSK,
77              .pmf_cfg = { .capable = true, .required = false },
78          },
79      };
80
81      ESP_ERROR_CHECK(esp_wifi_set_mode(WIFI_MODE_STA));
82      ESP_ERROR_CHECK(esp_wifi_set_config(WIFI_IF_STA, &wifi_config));
83      ESP_ERROR_CHECK(esp_wifi_start());
84
85      xEventGroupWaitBits(s_wifi_event_group, WIFI_CONNECTED_BIT,
86                          pdFALSE, pdTRUE, portMAX_DELAY);
87      ESP_LOGI(TAG, "Connected to hotspot: %s", WIFI_SSID);
88  }
89
90  static void udp_sender_task(void *pv) {
91      struct sockaddr_in dest = {
92          .sin_family      = AF_INET,
93          .sin_port        = htons(SEND_PORT),
94          .sin_addr.s_addr = inet_addr("255.255.255.255"),
95      };
96      int sock = socket(AF_INET, SOCK_DGRAM, IPPROTO_IP);
97      int bcast = 1;
98      setsockopt(sock, SOL_SOCKET, SO_BROADCAST, &bcast, sizeof(bcast));
99
100     char pkt[32];
101     uint32_t seq = 0;
102     while (1) {
103         snprintf(pkt, sizeof(pkt), "CSI:%lu", (unsigned long)seq++);
104         sendto(sock, pkt, strlen(pkt), 0,
105                (struct sockaddr *)&dest, sizeof(dest));
106         vTaskDelay(pdMS_TO_TICKS(SEND_INTERVAL_MS));
107     }
108 }
109
110 void app_main(void) {
111     ESP_ERROR_CHECK(nvs_flash_init());
112     wifi_init_sta();
113     xTaskCreate(udp_sender_task, "udp_tx", 4096, NULL, 5, NULL);
114     ESP_LOGI(TAG, "Sending at 50 Hz - ready.");
115 }
```

Listing 6.1: Complete csi_sender.c Code

6.2 ESP32 Receiver Code (csi_receiver_serial.c)

```c
#include <stdio.h>
#include <string.h>
#include <math.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/event_groups.h"
#include "esp_wifi.h"
#include "esp_event.h"
#include "nvs_flash.h"
#include "esp_log.h"
#include "esp_netif.h"
#include "esp_timer.h"

static const uint8_t SENDER_MAC[6] = {0xf4, 0x65, 0x0b, 0x54, 0xa6, 0x5c};
#define WIFI_SSID         "Google Pixel 8"
#define WIFI_PASS         "biteload"
#define TAG               "CSI_RECV"

#define WINDOW_SIZE    100
#define SUBCARRIER_NUM  52
#define CALIB_PACKETS   60
#define MOTION_RATIO   2.5f

static EventGroupHandle_t s_wifi_event_group;
#define WIFI_CONNECTED_BIT BIT0

static float csi_buf[WINDOW_SIZE][SUBCARRIER_NUM];
static int   buf_idx     = 0, buf_count = 0;
static float baseline    = 0.0f;
static int   calib_count = 0;
static bool  calibrated  = false;
static volatile uint32_t s_csi_packet_count = 0;
static esp_timer_handle_t s_reconnect_timer = NULL;

static int extract_amplitude(const int8_t *raw, int len, float *amp) {
    int n = 0;
    for (int i = 0; i + 1 < len && n < SUBCARRIER_NUM; i += 2)
        amp[n++] = sqrtf((float)raw[i] * raw[i] + (float)raw[i+1] * raw[i+1]);
    return n;
}

static float variance_1d(const float *a, int n) {
    if (n < 2) return 0.0f;
    float sum = 0.0f;
    for (int i = 0; i < n; i++) sum += a[i];
    float mean = sum / n, v = 0.0f;
    for (int i = 0; i < n; i++) { float d = a[i] - mean;
    v += d * d; }
    return v / n;
}

static float compute_motion_metric(void) {
    int cnt = (buf_count < WINDOW_SIZE) ? buf_count : WINDOW_SIZE;
    if (cnt < 5) return 0.0f;
    float total = 0.0f, col[WINDOW_SIZE];
    for (int s = 0; s < SUBCARRIER_NUM; s++) {
        for (int i = 0; i < cnt; i++) col[i] = csi_buf[i][s];
        total += variance_1d(col, cnt);
    }
    return total / SUBCARRIER_NUM;
}

static void wifi_csi_cb(void *ctx, wifi_csi_info_t *d) {
    if (!d || !d->buf || d->len < 4) return;
    if (memcmp(d->mac, SENDER_MAC, 6) != 0) return;

    float amp[SUBCARRIER_NUM];
    int n = extract_amplitude(d->buf, d->len, amp);
    if (n < 10) return;
```

```
    s_csi_packet_count++;
    memcpy(csi_buf[buf_idx % WINDOW_SIZE], amp, n * sizeof(float));
    buf_idx++;
    buf_count++;

    if (!calibrated) {
        if (buf_count < 5) return;
        float metric = compute_motion_metric();
        baseline = (baseline * calib_count + metric) / (calib_count + 1);
        calib_count++;
        if (calib_count % 10 == 0) {
            ESP_LOGI(TAG, "Calibrating %d/%d  metric=%.4f", calib_count,
                CALIB_PACKETS, metric);
            printf("CALIB,%d,%d,%.4f\n", calib_count, CALIB_PACKETS, metric);
            fflush(stdout);
        }
        if (calib_count >= CALIB_PACKETS) {
            calibrated = true;
            ESP_LOGI(TAG, "Calibrated! Baseline=%.4f", baseline);
            printf("CALIB_DONE,%.4f\n", baseline);
            fflush(stdout);
        }
        return;
    }

    float metric = compute_motion_metric();
    float ratio  = metric / (baseline + 1e-4f);
    bool  motion = (ratio > MOTION_RATIO);

    // Only adapt the baseline when the room is quiet
    if (!motion) {
        baseline = baseline * 0.98f + metric * 0.02f;
    }

    // Print the machine-readable line and flush immediately
    printf("DATA,%.4f,%.4f,%d\n", ratio, metric, motion ? 1 : 0);
    fflush(stdout);
}

static void reconnect_timer_cb(void *arg) {
    ESP_LOGI(TAG, "Retrying connection...");
    esp_wifi_connect();
}

static void wifi_event_handler(void *arg, esp_event_base_t event_base,
                               int32_t event_id, void *event_data) {
    if (event_base == WIFI_EVENT && event_id == WIFI_EVENT_STA_START) {
        esp_wifi_connect();
    } else if (event_base == WIFI_EVENT && event_id == WIFI_EVENT_STA_DISCONNECTED) {
        wifi_event_sta_disconnected_t *disc = (wifi_event_sta_disconnected_t
            *)event_data;
        ESP_LOGW(TAG, "Disconnected, reason=%d. Retrying in 5s...", disc->reason);
        esp_timer_start_once(s_reconnect_timer, 5000000);
        xEventGroupClearBits(s_wifi_event_group, WIFI_CONNECTED_BIT);
    } else if (event_base == IP_EVENT && event_id == IP_EVENT_STA_GOT_IP) {
        ip_event_got_ip_t *event = (ip_event_got_ip_t *)event_data;
        ESP_LOGI(TAG, "Got IP: " IPSTR, IP2STR(&event->ip_info.ip));
        xEventGroupSetBits(s_wifi_event_group, WIFI_CONNECTED_BIT);
        esp_timer_stop(s_reconnect_timer);
    }
}

static void wifi_init_sta(void) {
    s_wifi_event_group = xEventGroupCreate();
    ESP_ERROR_CHECK(esp_netif_init());
    ESP_ERROR_CHECK(esp_event_loop_create_default());
    esp_netif_create_default_wifi_sta();

    wifi_init_config_t cfg = WIFI_INIT_CONFIG_DEFAULT();
    ESP_ERROR_CHECK(esp_wifi_init(&cfg));

```

```
    const esp_timer_create_args_t timer_args = {
        .callback = reconnect_timer_cb,
        .name     = "wifi_reconnect"
    };
    ESP_ERROR_CHECK(esp_timer_create(&timer_args, &s_reconnect_timer));

    esp_event_handler_instance_t inst_any, inst_ip;
    ESP_ERROR_CHECK(esp_event_handler_instance_register(
        WIFI_EVENT, ESP_EVENT_ANY_ID, &wifi_event_handler, NULL, &inst_any));
    ESP_ERROR_CHECK(esp_event_handler_instance_register(
        IP_EVENT, IP_EVENT_STA_GOT_IP, &wifi_event_handler, NULL, &inst_ip));

    wifi_config_t wifi_config = {
        .sta = {
            .ssid     = WIFI_SSID,
            .password = WIFI_PASS,
            .threshold.authmode = WIFI_AUTH_WPA2_PSK,
            .pmf_cfg = { .capable = true, .required = false },
        },
    };

    ESP_ERROR_CHECK(esp_wifi_set_mode(WIFI_MODE_STA));
    ESP_ERROR_CHECK(esp_wifi_set_config(WIFI_IF_STA, &wifi_config));
    ESP_ERROR_CHECK(esp_wifi_start());

    xEventGroupWaitBits(s_wifi_event_group, WIFI_CONNECTED_BIT,
                        pdFALSE, pdTRUE, portMAX_DELAY);
    ESP_LOGI(TAG, "Connected to hotspot: %s", WIFI_SSID);
}

static void wifi_promiscuous_rx_cb(void *buf, wifi_promiscuous_pkt_type_t type) {}

static void csi_init(void) {
    wifi_promiscuous_filter_t filter = {
        .filter_mask = WIFI_PROMIS_FILTER_MASK_DATA |
        WIFI_PROMIS_FILTER_MASK_MGMT,
    };
    ESP_ERROR_CHECK(esp_wifi_set_promiscuous_filter(&filter));
    ESP_ERROR_CHECK(esp_wifi_set_promiscuous_rx_cb(wifi_promiscuous_rx_cb));
    ESP_ERROR_CHECK(esp_wifi_set_promiscuous(true));
    ESP_LOGI(TAG, "Promiscuous mode ON");

    ESP_ERROR_CHECK(esp_wifi_set_csi_rx_cb(wifi_csi_cb, NULL));
    wifi_csi_config_t config = {
        .lltf_en = true, .htltf_en = true, .stbc_htltf2_en = true,
        .ltf_merge_en = true, .channel_filter_en = false,
        .manu_scale = false, .shift = 0, .dump_ack_en = false,
    };
    ESP_ERROR_CHECK(esp_wifi_set_csi_config(&config));
    ESP_ERROR_CHECK(esp_wifi_set_csi(true));
    ESP_LOGI(TAG, "CSI ON - stand still for %d packets (calibration)", CALIB_PACKETS);
}

static void csi_debug_task(void *pv) {
    uint32_t last = 0;
    while (1) {
        vTaskDelay(pdMS_TO_TICKS(5000));
        uint32_t current = s_csi_packet_count;
        ESP_LOGI(TAG, "[DEBUG] CSI packets in last 5s: %lu | calib: %d/%d |
            calibrated: %s",
                 (unsigned long)(current - last), calib_count, CALIB_PACKETS,
                 calibrated ? "YES" : "NO");
        last = current;
    }
}

void app_main(void) {
    ESP_ERROR_CHECK(nvs_flash_init());
    wifi_init_sta();
    ESP_ERROR_CHECK(esp_wifi_set_ps(WIFI_PS_NONE));
    csi_init();
```

```
    xTaskCreate(csi_debug_task, "csi_dbg", 2048, NULL, 3, NULL);
}
```

Listing 6.2: Complete csi_receiver_serial.c Code

6.3 Python Backend Server (app.py)

```
# csi_dashboard/app.py
import serial
import threading
import time
import collections
import sys
import csv
from datetime import datetime
from flask import Flask, render_template, jsonify
from flask_socketio import SocketIO

app = Flask(__name__)
app.config['SECRET_KEY'] = 'csi_motion_key'
socketio = SocketIO(app, cors_allowed_origins="*", async_mode='threading')

SERIAL_PORT = "COM4"
BAUD_RATE = 115200
DEBOUNCE_TIME = 1.5  # seconds of quiet before motion event ends
BATCH_INTERVAL = 0.1  # emit to browser at 10 Hz (was 50 Hz - 5x reduction)

history = collections.deque(maxlen=300)
event_log = collections.deque(maxlen=50)

stats = {
    "total_packets": 0,
    "motion_events": 0,
    "packets_per_sec": 0.0,
    "in_motion": False,
    "calibrated": False,
    "baseline": 0.0,
    "calib_progress": 0,
    "serial_connected": False,
    "last_motion_ts": None,
}

# -- Batch buffer (filled by serial thread, drained by batch_emitter) -
_pending_batch = []
_batch_lock = threading.Lock()

# -- Rate counter ------------------------------------------------------
_rate_count = 0
_rate_ts = time.time()
_in_motion_since = None
_quiet_since = None

# -- CSV Recording Globals ---------------------------------------------
is_recording = False
csv_file = None
csv_writer = None


# -- Line parser -------------------------------------------------------
def handle_line(line: str):
    global _rate_count, _rate_ts, _in_motion_since, _quiet_since

    if line.startswith("CALIB,"):
        parts = line.split(",")
        if len(parts) == 4:
            stats["calib_progress"] = int(parts[1])
            socketio.emit("calib_progress", {
                "count": int(parts[1]),
                "total": int(parts[2]),
```

```
                "metric": float(parts[3]),
            })
        return

    if line.startswith("CALIB_DONE,"):
        parts = line.split(",")
        if len(parts) == 2:
            stats["calibrated"] = True
            stats["baseline"] = float(parts[1])
            socketio.emit("calib_done", {"baseline": float(parts[1])})
        return

    if not line.startswith("DATA,"):
        return

    parts = line.split(",")
    if len(parts) != 4:
        return

    try:
        ratio = float(parts[1])
        metric = float(parts[2])
        # Ignore the hardware flag and use the custom 1.8 threshold
        raw_motion = ratio >= 1.8
        ts = time.time()
    except ValueError:
        return

    # -- Packet rate ------------------------------------------------
    _rate_count += 1
    stats["total_packets"] += 1

    elapsed = ts - _rate_ts
    if elapsed >= 1.0:
        stats["packets_per_sec"] = round(_rate_count / elapsed, 1)
        _rate_count = 0
        _rate_ts = ts

    # -- Debounced motion state -------------------------------------
    if raw_motion:
        _quiet_since = None
        if not stats["in_motion"]:
            stats["in_motion"] = True
            stats["motion_events"] += 1
            _in_motion_since = ts
            entry = {"ts": ts, "event": "START",
                     "ratio": round(ratio, 2), "metric": round(metric, 4)}
            event_log.append(entry)
            socketio.emit("motion_event", entry)  # instant - rare event

        stats["last_motion_ts"] = ts
    else:
        if stats["in_motion"]:
            if _quiet_since is None:
                _quiet_since = ts
            if (ts - _quiet_since) > DEBOUNCE_TIME:
                duration = round(ts - _in_motion_since, 2) if _in_motion_since else 0
                entry = {"ts": ts, "event": "END", "duration": duration,
                         "ratio": round(ratio, 2), "metric": round(metric, 4)}
                event_log.append(entry)
                socketio.emit("motion_event", entry)  # instant - rare event

                stats["in_motion"] = False
                _in_motion_since = None
                _quiet_since = None

    # -- Push point into batch buffer (NOT emitting yet) ----------
    point = {
        "ts": round(ts, 3),
        "ratio": round(ratio, 3),
        "metric": round(metric, 4),
```

```
        "motion": stats["in_motion"],
    }
    history.append(point)
    with _batch_lock:
        _pending_batch.append(point)

    # -- Write to CSV if recording ----------
    global is_recording, csv_writer
    if is_recording and csv_writer:
        csv_writer.writerow([point["ts"], point["ratio"], point["metric"],
            int(point["motion"])])


# -- Batch emitter thread ---------------------------------------------
def batch_emitter():
    """
    Drains _pending_batch every BATCH_INTERVAL seconds and sends ONE
    WebSocket message to all browsers instead of one per serial line.
    Reduces browser render calls from 50/s to 10/s.
    """
    while True:
        time.sleep(BATCH_INTERVAL)
        with _batch_lock:
            if not _pending_batch:
                continue
            batch = list(_pending_batch)
            _pending_batch.clear()

        socketio.emit("csi_batch", {
            "points": batch,
            "stats": {
                "total_packets": stats["total_packets"],
                "motion_events": stats["motion_events"],
                "packets_per_sec": stats["packets_per_sec"],
                "in_motion": stats["in_motion"],
                "calibrated": stats["calibrated"],
                "baseline": round(stats["baseline"], 4),
                "calib_progress": stats["calib_progress"],
            },
        })


# -- Serial reader thread ---------------------------------------------
def serial_reader():
    while True:
        try:
            with serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=2) as ser:
                stats["serial_connected"] = True
                socketio.emit("serial_status", {"connected": True, "port":
                    SERIAL_PORT})
                print(f"[serial] Connected to {SERIAL_PORT} at {BAUD_RATE} baud")
                while True:
                    raw = ser.readline()

                    if not raw:
                        continue
                    handle_line(raw.decode("utf-8", errors="ignore").strip())
        except serial.SerialException as e:
            stats["serial_connected"] = False
            socketio.emit("serial_status", {"connected": False, "error": str(e)})
            print(f"[serial] Error: {e}. Retrying in 3 s...")
            time.sleep(3)


# -- HTTP routes ------------------------------------------------------
@app.route("/")
def index():
    return render_template("index.html")


@app.route("/api/history")
```

```python
def api_history():
    return jsonify(list(history))


@app.route("/api/stats")
def api_stats():
    return jsonify(stats)


@app.route("/api/events")
def api_events():
    return jsonify(list(event_log))


# -- SocketIO ------------------------------------------------------
@socketio.on("connect")
def on_connect():
    print("[ws] browser connected")
    socketio.emit("serial_status", {
        "connected": stats["serial_connected"],
        "port": SERIAL_PORT
    })
    socketio.emit("recording_status", {"recording": is_recording})


@socketio.on("disconnect")
def on_disconnect():
    print("[ws] browser disconnected")


@socketio.on("start_recording")
def handle_start_recording():
    global is_recording, csv_file, csv_writer
    if not is_recording:
        filename = f"motion_data_{datetime.now().strftime('%Y%m%d_%H%M%S')}.csv"
        csv_file = open(filename, 'w', newline='')
        csv_writer = csv.writer(csv_file)
        csv_writer.writerow(['timestamp', 'ratio', 'metric', 'in_motion'])

        is_recording = True
        socketio.emit("recording_status", {"recording": True, "file": filename})
        print(f"[recording] Started saving to {filename}")


@socketio.on("stop_recording")
def handle_stop_recording():
    global is_recording, csv_file, csv_writer
    if is_recording:
        is_recording = False
        if csv_file:
            csv_file.close()
            csv_file = None
        socketio.emit("recording_status", {"recording": False})
        print("[recording] Stopped saving.")


# -- Entry point ---------------------------------------------------
if __name__ == "__main__":
    if len(sys.argv) > 1:
        SERIAL_PORT = sys.argv[1]

    threading.Thread(target=serial_reader, daemon=True).start()
    threading.Thread(target=batch_emitter, daemon=True).start()

    print(f"[server] Dashboard -> http://localhost:5000")
    print(f"[server] Serial port: {SERIAL_PORT}")
    socketio.run(app, host="0.0.0.0", port=5000,
                 debug=False, allow_unsafe_werkzeug=True)
```

Listing 6.3: Complete app.py Code

6.4 Web Dashboard Frontend (index.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>CSI Motion Detector</title>
<script src="https://cdn.socket.io/4.7.5/socket.io.min.js"></script>
<script
    src="https://cdn.jsdelivr.net/npm/chart.js@4.4.3/dist/chart.umd.min.js"></script>
<style>
  :root {
    --bg:#0f1117;
    --surface:#1a1d27; --surface2:#222636;
    --border:#2e3347; --text:#e2e4f0; --muted:#7b80a0;
    --accent:#4f98a3; --green:#4caf82; --red:#e05c6a;
    --yellow:#e8af34; --radius:12px;
  }
  *{box-sizing:border-box;margin:0;padding:0;}
  body{background:var(--bg);color:var(--text);font-family:'Segoe
      UI',system-ui,sans-serif;
       font-size:14px;min-height:100vh;}

  header{display:flex;align-items:center;justify-content:space-between;
    padding:14px 24px;background:var(--surface);
    border-bottom:1px solid var(--border);position:sticky;top:0;z-index:10;}
  .logo{display:flex;align-items:center;gap:10px;font-weight:700;font-size:16px;}
  .logo svg{color:var(--accent);}
  #serial-badge{display:flex;align-items:center;gap:6px;font-size:12px;color:var(--muted);
    background:var(--surface2);border:1px solid var(--border);
    border-radius:99px;padding:4px 12px;}
  .dot{width:8px;height:8px;border-radius:50%;background:var(--red);transition:background
      .4s;}
  .dot.on{background:var(--green);animation:pulse 2s infinite;}
  @keyframes pulse{0%,100%{box-shadow:0 0 0 0 rgba(76,175,130,.5);}
                   50%{box-shadow:0 0 0 6px rgba(76,175,130,0);}}

  main{padding:20px 24px;display:grid;grid-template-columns:1fr 320px;
    gap:16px;max-width:1400px;margin:0 auto;}
  @media(max-width:900px){main{grid-template-columns:1fr;}}

  .card{background:var(--surface);border:1px solid var(--border);
        border-radius:var(--radius);padding:18px;}
  .card-title{font-size:11px;font-weight:600;letter-spacing:1px;
    text-transform:uppercase;color:var(--muted);margin-bottom:14px;}

  #motion-card{display:flex;flex-direction:column;align-items:center;
    justify-content:center;padding:32px;text-align:center;
    transition:background .4s,border-color .4s;}
  #motion-card.motion{background:rgba(224,92,106,.08);border-color:rgba(224,92,106,.4);}
  #motion-card.quiet{background:rgba(76,175,130,.05);border-color:rgba(76,175,130,.25);}
  #motion-icon{font-size:56px;line-height:1;margin-bottom:8px;transition:transform
      .2s;}
  #motion-card.motion #motion-icon{transform:scale(1.1);}
  #motion-label{font-size:26px;font-weight:800;letter-spacing:1px;
    margin-bottom:4px;transition:color .3s;}
  #motion-card.motion #motion-label{color:var(--red);}
  #motion-card.quiet  #motion-label{color:var(--green);}
  #motion-sub{font-size:12px;color:var(--muted);}

  .stats-row{display:grid;grid-template-columns:repeat(4,1fr);gap:12px;}
  @media(max-width:700px){.stats-row{grid-template-columns:repeat(2,1fr);}}
  .stat{background:var(--surface);border:1px solid var(--border);
    border-radius:var(--radius);padding:14px 16px;}
  .stat-label{font-size:11px;color:var(--muted);margin-bottom:6px;
    text-transform:uppercase;letter-spacing:.8px;}
  .stat-value{font-size:24px;font-weight:700;font-variant-numeric:tabular-nums;}
  .stat-value.accent{color:var(--accent);}
  .stat-value.red{color:var(--red);}
  .stat-value.yellow{color:var(--yellow);}
```

```
66
67    .chart-wrap{position:relative;height:240px;}
68
69    #calib-banner{background:rgba(232,175,52,.1);border:1px solid rgba(232,175,52,.35);
70      border-radius:var(--radius);padding:14px 18px;display:flex;
71      align-items:center;gap:12px;font-size:13px;}
72    #calib-banner.hidden{display:none;}
73    #calib-bar-wrap{flex:1;height:6px;background:var(--border);border-radius:3px;overflow:hidden;}
74    #calib-bar{height:100%;background:var(--yellow);width:0%;transition:width
          .3s;border-radius:3px;}
75
76    .right-col{display:flex;flex-direction:column;gap:16px;}
77
78    #ratio-value{font-size:52px;font-weight:800;font-variant-numeric:tabular-nums;
79      text-align:center;margin:8px 0 4px;transition:color .3s;}
80    .threshold-line{display:flex;justify-content:space-between;align-items:center;
81      font-size:12px;color:var(--muted);padding:8px 0;border-top:1px solid
            var(--border);}
82    .threshold-pill{background:rgba(224,92,106,.15);color:var(--red);
83      border-radius:99px;padding:2px 8px;font-size:11px;}
84
85    #event-log{max-height:260px;overflow-y:auto;}
86    #event-log::-webkit-scrollbar{width:4px;}
87    #event-log::-webkit-scrollbar-thumb{background:var(--border);border-radius:2px;}
88    .log-item{display:flex;align-items:flex-start;gap:10px;padding:8px 0;
89      border-bottom:1px solid var(--border);font-size:12px;
90      animation:fadeIn .3s ease;}
91    .log-item:last-child{border-bottom:none;}
92    @keyframes fadeIn{from{opacity:0;transform:translateX(-8px);}to{opacity:1;}}
93    .log-badge{border-radius:4px;padding:2px 7px;font-size:10px;font-weight:700;
94      white-space:nowrap;flex-shrink:0;}
95    .log-badge.start{background:rgba(224,92,106,.2);color:var(--red);}
96    .log-badge.end{background:rgba(76,175,130,.15);color:var(--green);}
97    .log-text{color:var(--muted);line-height:1.5;}
98    .log-text strong{color:var(--text);}
99    .empty-log{text-align:center;padding:24px;color:var(--muted);font-size:12px;}
100 </style>
101 </head>
102 <body>
103
104 <header>
105   <div class="logo">
106     <svg width="24" height="24" viewBox="0 0 24 24" fill="none"
107          stroke="currentColor" stroke-width="2">
108       <path d="M1 12s4-7 11-7 11 7 11 7-4 7-11 7-11-7-11-7z"/>
109       <circle cx="12" cy="12" r="3"/>
110     </svg>
111     CSI Motion Detector
112   </div>
113
114   <div style="display:flex; align-items:center; gap:16px; margin-left:auto;
          margin-right:20px;">
115     <span id="record-status" style="font-size:12px; color:var(--muted);"></span>
116     <button id="record-btn" style="background:var(--surface2); color:var(--text);
            border:1px solid var(--border); padding:6px 16px; border-radius:99px;
            cursor:pointer;
117       font-weight:600; display:flex; align-items:center; gap:8px; transition: 0.3s;">
118       <span id="record-dot" style="width:10px; height:10px; border-radius:50%;
119       background:var(--red);"></span>
120       <span id="record-btn-text">Start Recording</span>
121     </button>
122   </div>
123
124   <div id="serial-badge">
125     <span class="dot" id="serial-dot"></span>
126     <span id="serial-label">Connecting...</span>
127   </div>
128 </header>
129
130 <main>
131   <div style="display:flex;flex-direction:column;gap:16px;">
```

```
132
133     <div id="calib-banner">
134       <span>   </span>
135       <div style="flex:1">
136         <div style="margin-bottom:6px">
137           Calibrating - <strong>stand still</strong>
138           <span id="calib-text">0 / 60</span>
139         </div>
140
141         <div id="calib-bar-wrap"><div id="calib-bar"></div></div>
142       </div>
143     </div>
144
145     <div class="stats-row">
146       <div class="stat">
147         <div class="stat-label">Packets / sec</div>
148         <div class="stat-value accent" id="stat-rate">-</div>
149       </div>
150       <div class="stat">
151         <div class="stat-label">Total Packets</div>
152         <div class="stat-value" id="stat-total">0</div>
153       </div>
154       <div class="stat">
155
156         <div class="stat-label">Motion Events</div>
157         <div class="stat-value red" id="stat-events">0</div>
158       </div>
159       <div class="stat">
160         <div class="stat-label">Baseline</div>
161         <div class="stat-value yellow" id="stat-baseline">-</div>
162       </div>
163     </div>
164
165     <div class="card">
166       <div class="card-title">Ratio Over Time - <span
              style="color:rgba(224,92,106,.8)">- -</span> threshold 1.8</div>
167       <div class="chart-wrap"><canvas id="ratioChart"></canvas></div>
168     </div>
169
170     <div class="card">
171
172        <div class="card-title">Motion Metric (mean temporal variance)</div>
173       <div class="chart-wrap"><canvas id="metricChart"></canvas></div>
174     </div>
175
176   </div>
177
178   <div class="right-col">
179     <div class="card" id="motion-card">
180       <div id="motion-icon">    </div>
181       <div id="motion-label">QUIET</div>
182       <div id="motion-sub">Waiting for data...</div>
183     </div>
184
185     <div class="card">
186       <div class="card-title">Current Ratio</div>
187       <div id="ratio-value" style="color:var(--accent)">-</div>
188
189        <div class="threshold-line">
190         <span>Motion threshold</span>
191
192         <span class="threshold-pill">&ge; 1.8</span>
193       </div>
194     </div>
195
196     <div class="card" style="flex:1">
197       <div class="card-title">Motion Event Log</div>
198       <div id="event-log"><div class="empty-log">No events yet</div></div>
199     </div>
200   </div>
201 </main>
```

```
<script>
// -- Constants ------------------------------------------------------
const CHART_MAX_POINTS = 150;
const THRESHOLD        = 1.8;
// -- Chart: common scale / plugin options ---------------------------
const commonOpts = (yLabel) => ({
  responsive: true,
  maintainAspectRatio: false,
  animation: false,
  normalized: true,
  interaction: { mode: 'index', intersect: false },
  plugins: { legend: { display: false },
             tooltip: { backgroundColor:'#1a1d27', borderColor:'#2e3347',
                        borderWidth:1, titleColor:'#7b80a0', bodyColor:'#e2e4f0' }},
  scales: {
    x: { display: false },
    y:
       { grid: { color:'rgba(46,51,71,.6)' },
         ticks: { color:'#7b80a0', font:{ size:11 } },
         title: { display:true, text:yLabel, color:'#7b80a0', font:{ size:11 } } }
  }
});
// -- Ratio chart ----------------------------------------------------
const ratioCtx = document.getElementById('ratioChart').getContext('2d');
const ratioChart = new Chart(ratioCtx, {
  type: 'line',
  data: {
    labels: [],
    datasets: [
      {
        label: 'Ratio',
        data: [],
        borderColor: '#4f98a3',
        backgroundColor: 'rgba(79,152,163,.08)',
        borderWidth: 2, pointRadius: 0, tension: 0.3, fill: true,
        order: 1,

       },
      {
        label: 'Threshold',
        data: [],
        borderColor: 'rgba(224,92,106,.65)',
        borderWidth: 1.5,
        borderDash: [6, 4],
        pointRadius: 0,
        fill: false,
        tension: 0,
        order: 2,

  }
    ]

   },
  options: commonOpts('ratio'),
});
// -- Metric chart ---------------------------------------------------
const metricCtx = document.getElementById('metricChart').getContext('2d');
const metricChart = new Chart(metricCtx, {
  type: 'line',
  data: {
    labels: [],
    datasets: [{
      label: 'Metric',
      data: [],
      borderColor: '#e8af34',
      backgroundColor: 'rgba(232,175,52,.07)',
      borderWidth: 2, pointRadius: 0, tension: 0.3, fill: true,
    }]
  },
  options: commonOpts('variance'),
```

```
});
// -- Efficient batch push ------------------------------------------
function pushBatch(ratioPoints, metricPoints, labels) {
  ratioChart.data.labels.push(...labels);
  ratioChart.data.datasets[0].data.push(...ratioPoints);
  ratioChart.data.datasets[1].data.push(...ratioPoints.map(() => THRESHOLD));

  metricChart.data.labels.push(...labels);
  metricChart.data.datasets[0].data.push(...metricPoints);
  const trim = (chart, maxPts) => {
    const excess = chart.data.labels.length - maxPts;
    if (excess > 0) {
      chart.data.labels.splice(0, excess);
      chart.data.datasets.forEach(ds => ds.data.splice(0, excess));
    }
  };
  trim(ratioChart,  CHART_MAX_POINTS);
  trim(metricChart, CHART_MAX_POINTS);

  ratioChart.update('none');
  metricChart.update('none');
}

// -- UI helpers ----------------------------------------------------
function fmtTime(ts) {
  const d = new Date(ts * 1000);
  return d.toLocaleTimeString('en-IN', { hour12:false }) + '.'
       + String(d.getMilliseconds()).padStart(3,'0');
}

function colourForRatio(r) {
  if (r >= THRESHOLD) return 'var(--red)';
  if (r >= 1.5)       return 'var(--yellow)';
  return 'var(--accent)';
}

// updateUI is called once per batch (last point wins) ------------------
function updateUI(lastPoint, s) {
  const card  = document.getElementById('motion-card');
  const icon  = document.getElementById('motion-icon');
  const label = document.getElementById('motion-label');
  const sub   = document.getElementById('motion-sub');

  const isMotion = lastPoint.motion;
  card.className  = 'card ' + (isMotion ? 'motion' : 'quiet');
  icon.textContent  = isMotion ? '    ' : '    ';
  label.textContent = isMotion ? 'MOTION' : 'QUIET';
  sub.textContent   = `ratio: ${lastPoint.ratio.toFixed(2)}`;

  const rv = document.getElementById('ratio-value');
  rv.textContent = lastPoint.ratio.toFixed(3);
  rv.style.color = colourForRatio(lastPoint.ratio);

  document.getElementById('stat-rate').textContent   = s.packets_per_sec;
  document.getElementById('stat-total').textContent  =
      s.total_packets.toLocaleString();
  document.getElementById('stat-events').textContent = s.motion_events;
  if (s.baseline) document.getElementById('stat-baseline').textContent =
      s.baseline.toFixed(4);
  if (!s.calibrated)
      document.getElementById('calib-banner').classList.remove('hidden');
}

// -- Load history on page open -------------------------------------
fetch('/api/history').then(r => r.json()).then(pts => {
  if (!pts.length) return;
  const labels  = pts.map(p => fmtTime(p.ts));
  const ratios  = pts.map(p => p.ratio);
  const metrics = pts.map(p => p.metric);
  pushBatch(ratios, metrics, labels);
});
```

```
// -- SocketIO ------------------------------------------------------------
const socket = io();

socket.on('serial_status', d => {
  document.getElementById('serial-dot').classList.toggle('on', d.connected);
  document.getElementById('serial-label').textContent =
    d.connected ? `${d.port} - Connected`
                : (d.error ? `Disconnected: ${d.error}` : 'Disconnected');
});
socket.on('calib_progress', d => {
  document.getElementById('calib-text').textContent = `${d.count} / ${d.total}`;
  document.getElementById('calib-bar').style.width = `${(d.count/d.total)*100}%`;
});
socket.on('calib_done', d => {
  document.getElementById('calib-banner').classList.add('hidden');
  document.getElementById('stat-baseline').textContent = d.baseline.toFixed(4);
});
socket.on('csi_batch', d => {
  const pts     = d.points;
  if (!pts || !pts.length) return;

  const labels  = pts.map(p => fmtTime(p.ts));
  const ratios  = pts.map(p => p.ratio);
  const metrics = pts.map(p => p.metric);

  pushBatch(ratios, metrics, labels);
  updateUI(pts[pts.length - 1], d.stats);
});
socket.on('motion_event', e => {
  const log   = document.getElementById('event-log');
  const empty = log.querySelector('.empty-log');
  if (empty) empty.remove();

  const item = document.createElement('div');
  item.className = 'log-item';
  if (e.event === 'START') {
    item.innerHTML = `
      <span class="log-badge start">START</span>
      <div class="log-text"><strong>${fmtTime(e.ts)}</strong><br>
      ratio ${e.ratio} - metric ${e.metric}</div>`;
  } else {
    item.innerHTML = `
      <span class="log-badge end">END</span>
      <div class="log-text"><strong>${fmtTime(e.ts)}</strong><br>
      Duration: ${e.duration}s - ratio
      ${e.ratio}</div>`;
  }
  log.prepend(item);
  if (log.children.length > 30) log.lastElementChild.remove();
});
// -- CSV Recording Controls ----------------------------------------------
let isRecording = false;
const recordBtn = document.getElementById('record-btn');
const recordBtnText = document.getElementById('record-btn-text');
const recordDot = document.getElementById('record-dot');
const recordStatus = document.getElementById('record-status');
recordBtn.addEventListener('click', () => {
  if (isRecording) {
    socket.emit('stop_recording');
  } else {
    socket.emit('start_recording');
  }
});
socket.on('recording_status', d => {
  isRecording = d.recording;
  if (isRecording) {
    recordBtn.style.borderColor = 'var(--red)';
    recordBtn.style.background = 'rgba(224,92,106,.1)';
    recordDot.style.animation = 'pulse 2s infinite';
    recordBtnText.textContent = 'Stop Recording';
    recordStatus.textContent = `Saving to ${d.file}`;
```

```
412   } else {
413     recordBtn.style.borderColor = 'var(--border)';
414     recordBtn.style.background = 'var(--surface2)';
415     recordDot.style.animation = 'none';
416     recordBtnText.textContent = 'Start Recording';
417     recordStatus.textContent = 'Saved successfully.';
418
419     setTimeout(() => {
420         if (!isRecording) recordStatus.textContent =
421         '';
422     }, 4000);
423   }
424 });
425 </script>
426 </body>
427 </html>
```

Listing 6.4: Complete index.html Code

7. Results

Our custom variance-based metric proved to be much more reliable than tracking raw amplitude.

- **Stable State:** When the room was empty, the variance ratio hovered perfectly around **1.0**. The baseline adaptation algorithm successfully ignored slow environmental drifts.
- **Motion State:** When a human walked between the ESP32 antennas, multipath fading heavily disrupted the 52 subcarriers. The variance ratio spiked instantly to values between **2.5 and 5.0**.
- **Filtering:** By implementing a strict threshold of ≥ 1.8 paired with a 1.5-second debounce timer in our Python script, we completely eliminated false-positive "flickers" that usually plague raw RF data.

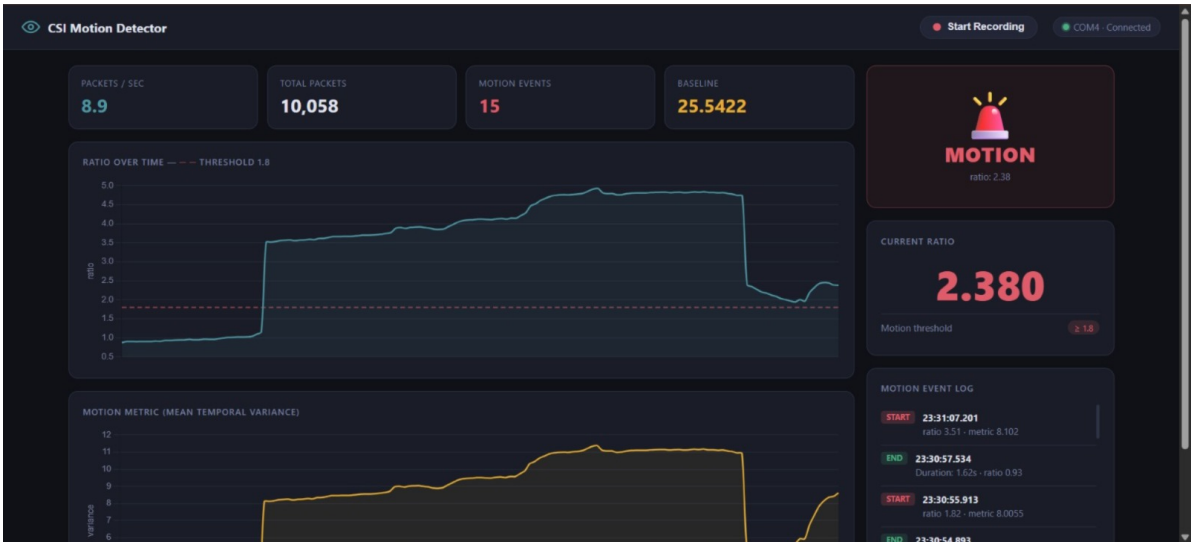

Figure 7.1: Browser dashboard — Top view showing statistics and motion detection state

Figure 7.2: Browser dashboard — Chart view tracking variance ratio and metrics over time

7.1 Illustrative Ratio Time-Series

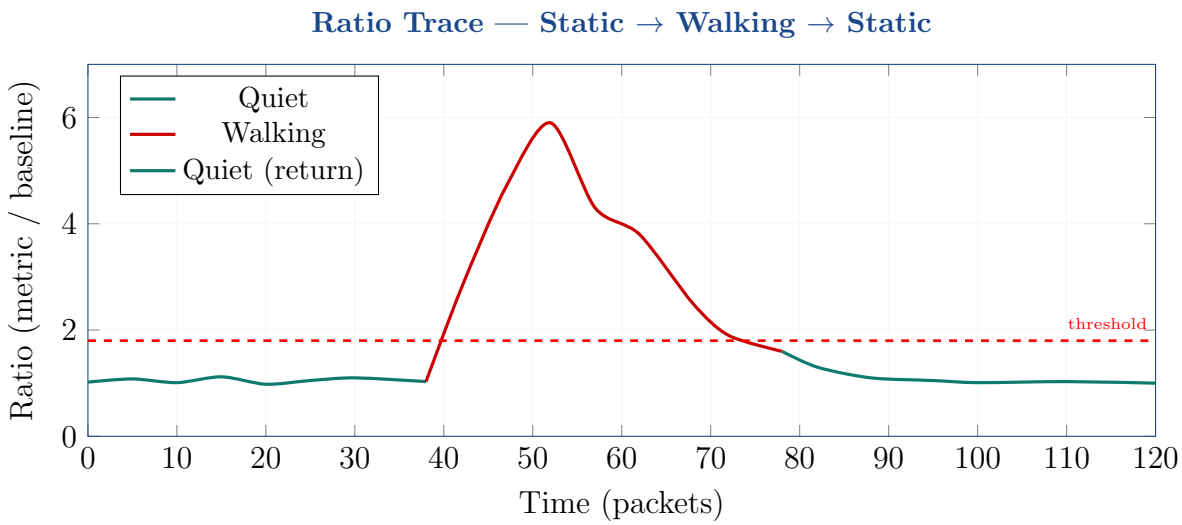

Figure 7.3: Representative ratio time-series from the browser dashboard

8. Conclusions

The project was highly successful. A complete real-time human activity detection system using Wi-Fi CSI data captured from ESP32 microcontrollrs is built. The system successfully distinguishes between two states - Quiet (no activity detection) and Motion (activity detection).

9. Future Scopes

- **Machine Learning Integration:** While the current variance metric reliably detects binary motion, applying advanced algorithms (e.g., SVM or CNN) to the raw CSI data could classify specific activities, allowing the system to distinguish between a human walking and environmental noise (e.g., a fan spinning).
- **Phase Data Utilization:** The current implementation extracts only I/Q magnitude (amplitude). Expanding the processing pipeline to factor in CSI phase shifts would provide valuable directional data and improve the granularity of movement detection.
- **Multi-Node Spatial Tracking:** Scaling the current dual-node setup into a larger mesh network (e.g., 4 to 5 ESP32 modules placed in room corners) would allow for precise 2D spatial tracking and exact coordinate mapping of a person inside a room.
- **Elderly-Care Applications (Fall Detection):** A simple state machine could be added to the `wifi_csi_cb` callback to detect sharp, high-amplitude CSI transients followed by complete stillness. This would enable the system to act as a privacy-preserving fall detection monitor for elderly care.